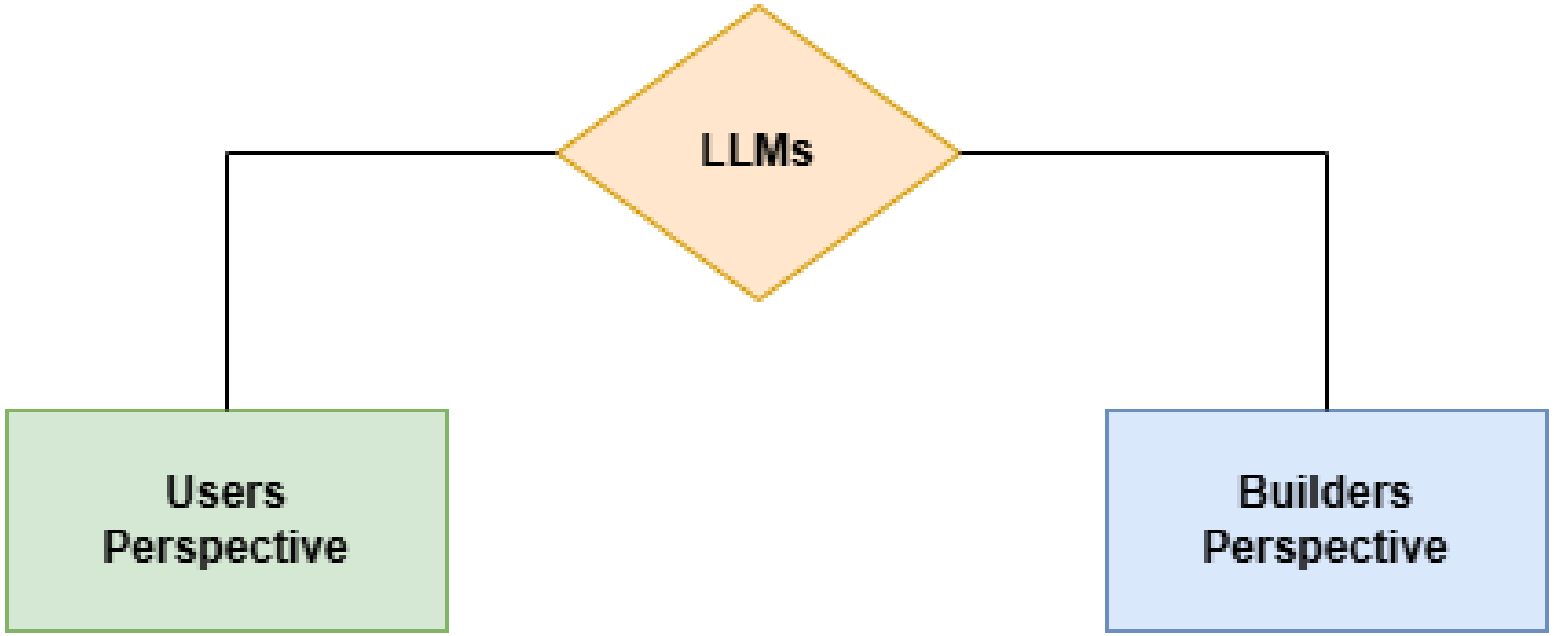
University at Buffalo
1Capen

Mastering LLMs: From Fundamentals to Advanced

Part 2 : Practical LLM Engineering

LLMs Engineering Stack



Quantization: Why we do it

LLMs are powerful but expensive to run

Quantization reduces memory and speeds up inference

Lets you run larger models on the same hardware budget

Key idea: represent weights in fewer bits with minimal quality loss
Visual: “Model size and latency” before vs after chart (simple bars)

What actually changes in quantization

- Original weights are stored in high precision (FP16 or FP32)
- Quantization stores weights in lower precision (INT8 or INT4)
- Computation can be faster and memory bandwidth drops a lot
- Tradeoff: small accuracy drop possible, especially on hard tasks

Visual: FP16 to INT8 to INT4 ladder with “cost” and “quality” arrows

Common quantization levels and tradeoffs

- INT8: good balance for many inference workloads
- INT4: best compression, bigger risk of quality drop
- Rule of thumb: quantize for serving, not for learning new knowledge

Visual: table like layout in slide with “quality, cost, speed” icons

- FP16: strong quality, high memory

When quantization helps most

- When you need lower latency for interactive apps
- When you want to fit model on limited GPU or even CPU
- When you are deploying endpoints and cost matters
- When you want to fine tune cheaply using QLoRA

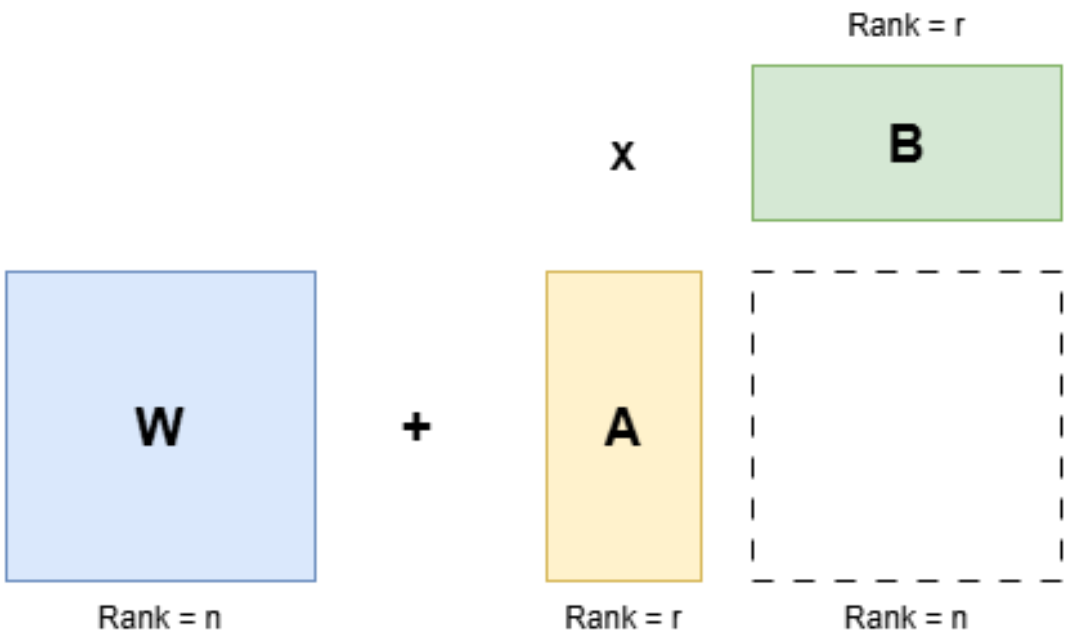
Fine tuning in 60 seconds

- Fine tuning changes model behavior using your data
- Helps when you want consistent style, format, or domain behavior
- But full fine tuning is expensive and can overfit small datasets
- Efficient fine tuning is the practical default in 2026

LoRA intuition (why it works)

- Instead of updating all weights, add small trainable adapter matrices
- Base model stays frozen, adapters learn task specific changes
- Faster training, less VRAM, easier experimentation
- Great for instruction style tuning or domain adaptation

LoRA intuition (why it works)



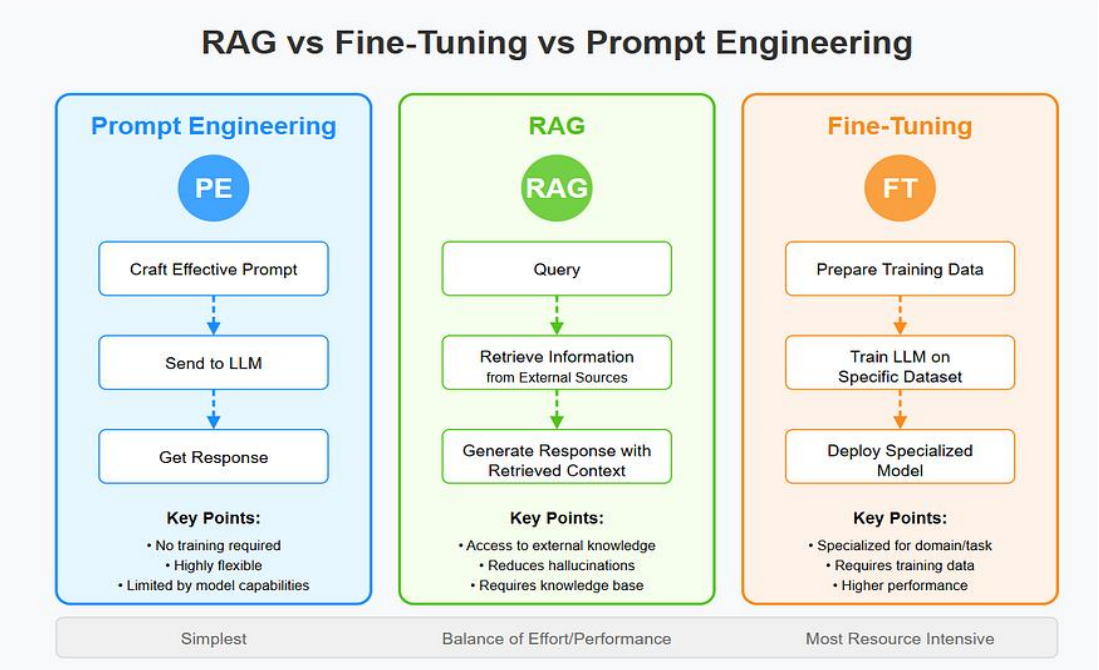
$W = W0 + \gamma(AB)$ $\gamma = \alpha/r$

QLoRA in one slide

•Common for open source models when resources are limited
Visual: Pipeline: Quantize base → Add adapters → Train adapters
•QLoRA = quantize the base model + train LoRA adapters
•Big cost reduction while still learning useful behavior changes
•Makes fine tuning possible on smaller GPUs

When to prompt vs fine tune vs RAG

- Prompting: fastest to start, good for general tasks
- Fine tuning: best when you want consistent behavior and format
- RAG: best when you need factual grounding, private docs, or freshness
- In real products, you often combine RAG + light tuning



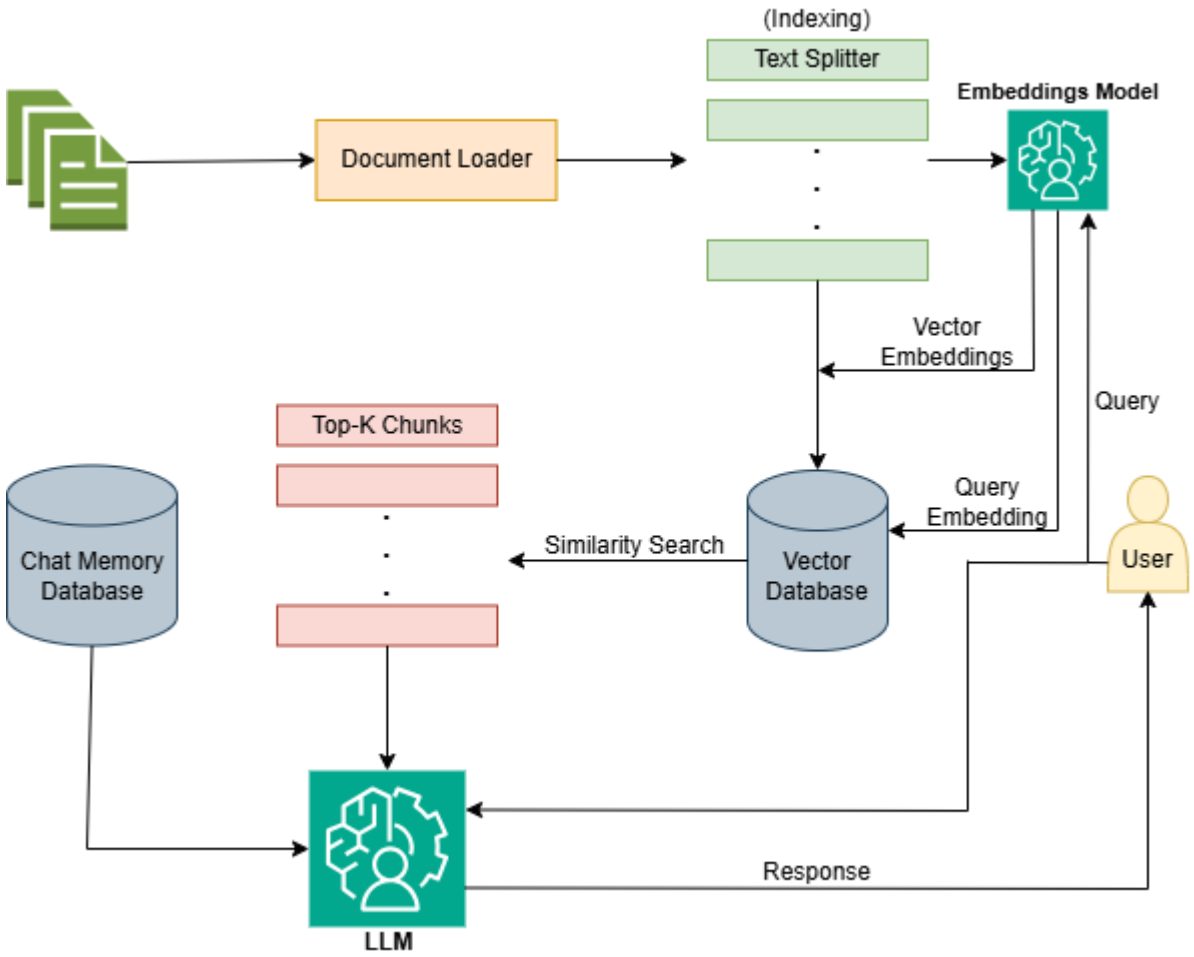
Why RAG became the norm

- You cannot keep fine tuning every time knowledge changes
- Easier updates: change documents, not model weights
- RAG keeps answers grounded in your sources
- Better for enterprise use cases with private data and citations

RAG, explained as a simple loop

- User asks a question
- System retrieves relevant chunks from your data
- LLM answers using retrieved context
- Goal: accurate answers that cite and stick to the source

RAG, explained as a simple loop



Key RAG terms you must know

- Chunking: split documents into searchable pieces
- Embeddings: convert text to vectors for similarity search
- Vector store: database optimized for nearest neighbor search
- Retriever: gets top k relevant chunks
- Context assembly: builds the final prompt for the LLM

Chunking decisions that change everything

•Chunk size controls recall vs precision
•Bad chunking is a common reason RAG fails
Visual: doc page split into chunks with overlap show
•Add metadata like title, section, page number
•Overlap helps preserve context across boundaries

Embeddings and vector storage

- Vector DB stores vectors and metadata and supports top k search
- Embedding model maps text to a vector space
- Retrieval quality depends on embedding choice and indexing settings

Visual: 2D embedding scatter plot concept

- Similar questions and relevant chunks should be close in that space

Retrieval: what “top k” really means

- Retriever returns top k chunks by similarity score
- Filters help: metadata filters, document scope, date range
- Garbage in retrieval means garbage out generation

Visual: Top k list with a reranker reordering results

- Reranking can improve ordering using a stronger scoring model

Baseline RAG demo plan (what we will build now)

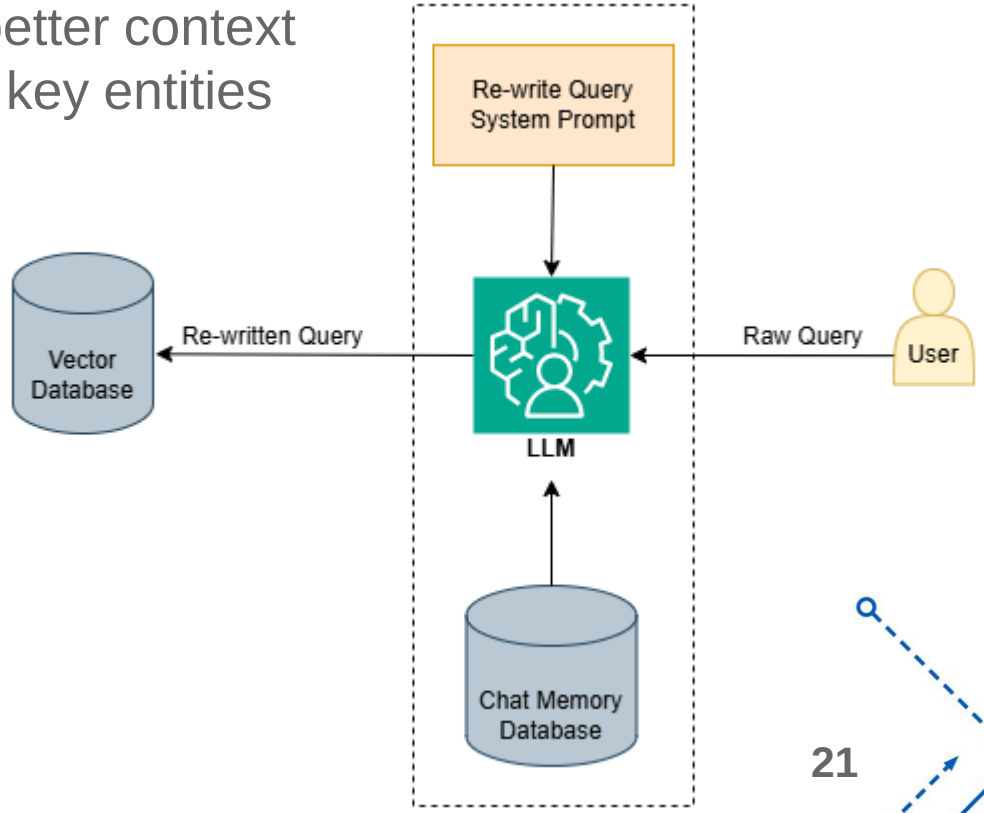
- Load a small document set
- Chunk documents
- Create embeddings
- Store in vector DB
- Retrieve top k for a query
- Generate answer using retrieved context

Common failure modes of baseline RAG

- Missing the right chunk due to wording mismatch
- Chunks too big or too small
- Embeddings not ideal for your domain
- LLM answers confidently even with weak context
- High latency from too many retrieved chunks

One simple upgrade: Query rewriting

- Before retrieving, use a small LLM to rewrite the user query
- Goal: make the query more specific and retrieval friendly
- Then retrieve using rewritten query and answer with better context
- Examples: add synonyms, expand acronyms, include key entities



Metrics we will compare (simple and practical)

- Retrieval hit rate: did we fetch the right chunk
- Faithfulness check: is the answer supported by retrieved text
- Latency: baseline vs rewritten pipeline time
- Recall at k: does the correct source appear in top k

What is next in RAG in 2026 (quick tour)

- Hybrid retrieval: BM25 + embeddings together
- Reranking becomes standard for quality
- Better chunking: structure aware splitting, semantic chunking
- Context compression and citations for long documents
- Evaluation driven iteration with test sets and monitoring

Multi Agentic AI

- Agentic AI means multiple steps with planning, tools, and verification
- Common pattern: planner agent, retriever agent, writer agent, critic agent
- Used when tasks require tool calls, multi step reasoning, or workflows
- Frameworks: LangChain, LangGraph, and no code tools like n8n

Resources

- Github Repo Link for Part-2 of workshop

References

•https://jillanisofttech.medium.com/rag-vs-fine-tuning-vs-prompt-engineering-optimizing-ai-llm-models-9073013e0662